الجزائريـــــــة الديمقراطيـــــة الشعبيـــة
وزارة التعليـــــم العـــــالي و البحـــــث العلـــمي
جامعة وهران للعلوم والتكنولوجيا محمد بوضياف
كلية الريـــاضيـــات و اللإعـــلام الآلـــي
République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur Et de la Recherche Scientifique
Université des Sciences et de la Technologie d'Oran Mohamed BOUDIAF
Faculté des Mathématiques et Informatique
Département D'Informatique

Master1 ML DS-IA **Fiche TP3** **Arbres de décision:** *Prédiction du risque de crédit*

Dr. Ourdighi A.

Objectif

À l'issue de ce TP, vous devrez être capable de :

- *Comprendre l'anatomie d'un arbre de décision : nœud interne, branche, feuille, racine, profondeur.*
- *Prétraiter un jeu de données réel : valeurs manquantes, variables quantitatives (numériques) et catégorielles.*
- *Construire, visualiser et évaluer un arbre de décision (scikit-learn).*
- *Lire l'arbre comme un ensemble de règles « SI … ALORS … » et analyser le sur-apprentissage.*

2. Dataset

2.1. Jeu de données fourni : Dataset credit tp3 · CSV

Une banque souhaite automatiser une première décision sur les demandes de crédit. Pour chaque client, on dispose de caractéristiques financières et on cherche à prédire la décision : Acceptation, Refus ou Risque élevé. Le jeu de données est fictif et ne constitue pas un conseil financier.

Variable	Type	Description
id_client	Identifiant	Numéro du client (à exclure des variables explicatives)
age	Quantitative discrète	Âge du client (années)
revenu_annuel	Quantitative continue	Revenu annuel
anciennete_emploi	Quantitative discrète	Ancienneté dans l'emploi actuel (années)
dette_totale	Quantitative continue	Montant total des dettes
ratio_dette_revenu	Quantitative continue	dette_totale / revenu_annuel
nb_credits_en_cours	Quantitative discrète	Nombre de crédits en cours
retards_paiement	Quantitative discrète	Nombre de retards de paiement passés
revenu_stable	Qualitative binaire	Oui / Non
decision (cible)	Qualitative (3 classes)	Acceptation / Refus / Risque élevé

3. Travail Demandé

Partie A : Exploration des données

1. Charger le fichier avec pandas.read_csv, afficher les 5 premières lignes, la forme (shape) et les types (info()).
2. Quelles variables sont quantitatives ? Lesquelles sont qualitatives ? Justifier.
3. Afficher la répartition de la cible (value_counts). Le jeu est-il équilibré ?
4. Calculer les statistiques descriptives (describe()) des variables quantitatives : moyenne, médiane, écart-type, min, max.

Partie B : Valeurs manquantes et données quantitatives

Rappel : une valeur manquante (NaN) est une information absente. La plupart des algorithmes de scikit-learn ne l'acceptent pas : il faut la traiter avant l'apprentissage. Les variables quantitatives peuvent aussi contenir des valeurs aberrantes (outliers) qui faussent les statistiques.

1. Détecter les valeurs manquantes

```
df.isna().sum()

(df.isna().mean() * 100).round(1)   # pourcentage par colonne
```

1. Quelles colonnes contiennent des valeurs manquantes ? Quel est le pourcentage pour chacune ? Représenter les résultats par un diagramme en barres.

2. Stratégies de traitement

Stratégie	Principe	Quand l'utiliser ?
Suppression des lignes	dropna()	Peu de lignes touchées (< 5 %)
Imputation par la moyenne	fillna(df[c].mean())	Variable quantitative symétrique, sans outliers
Imputation par la médiane	fillna(df[c].median())	Variable quantitative avec outliers ou asymétrique
Imputation par le mode	fillna(df[c].mode()[0])	Variable qualitative

2. Pour chaque colonne quantitative ayant des valeurs manquantes, calculer moyenne et médiane. Les deux sont-elles proches ? Pour quelle colonne l'écart est-il le plus grand et pourquoi ?

3. Appliquer une stratégie adaptée à chaque colonne (quantitative : médiane ou moyenne ; qualitative revenu_stable : mode). Justifier chaque choix.

4. Pour ratio_dette_revenu, comparer deux méthodes : (i) imputation par la médiane ; (ii) recalcul exact à partir de dette_totale / revenu_annuel lorsque ces deux colonnes sont disponibles. Laquelle est la plus fiable ?

3. Données quantitatives : valeurs aberrantes et échelle

5. Tracer un boxplot de age et de revenu_annuel. Repérer les valeurs aberrantes (indice : un âge impossible et un revenu extrême). Les traiter (suppression, remplacement par NaN puis imputation, ou écrêtage via la règle IQR : Q1 − 1,5·IQR ; Q3 + 1,5·IQR).

6. Vérifier la cohérence : ratio_dette_revenu ≥ 0, age ≥ 18, anciennete_emploi ≤ age − 16.

7. Question de réflexion : les arbres de décision nécessitent-ils une normalisation des variables quantitatives (standardisation, min-max) ? Pourquoi, contrairement au perceptron ou aux k plus proches voisins ?

8. Encoder la variable qualitative revenu_stable en 0/1 (map ou get_dummies), puis séparer X (variables explicatives, sans id_client) et y (decision).

Partie C : Construction de l'arbre

1. Découper les données : 70 % apprentissage / 30 % test, avec stratification sur la cible et random_state=42 (train_test_split).

2. Entraîner un DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42). Afficher l'arbre avec plot_tree ou export_text.

3. Identifier sur l'arbre obtenu : la racine, un nœud interne, une branche, une feuille. Donner la profondeur et le nombre de feuilles.

4. Quelle variable est choisie à la racine ? Expliquer (impureté de Gini / gain d'information). Comparer avec criterion='entropy'.

5. Écrire au moins trois règles de décision sous la forme « SI … ET … ALORS … » en lisant les chemins racine → feuille. Exemple : SI ratio_dette_revenu ≤ 0,4 ET revenu_stable = Oui ALORS Acceptation.

6. Prédire la décision pour un nouveau client : age = 35, revenu_annuel = 55 000, anciennete_emploi = 6, ratio_dette_revenu = 0,30, nb_credits_en_cours = 1, retards_paiement = 0, revenu_stable = Oui. Suivre manuellement le chemin dans l'arbre et vérifier avec predict().

Partie D : Évaluation et sur-apprentissage

1. Calculer l'exactitude (accuracy) sur l'apprentissage et sur le test, la matrice de confusion et le rapport de classification (precision, recall, F1).

2. Faire varier max_depth de 1 à 15. Tracer les courbes d'exactitude apprentissage et test en fonction de la profondeur. À partir de quelle profondeur observe-t-on du sur-apprentissage ?

3. Afficher feature_importances_ et classer les variables. Les variables les plus importantes sont-elles cohérentes avec l'intuition métier ?

4. Quel effet ont min_samples_leaf et max_depth sur la taille de l'arbre et sa capacité de généralisation ? Proposer un meilleur jeu d'hyperparamètres (GridSearchCV, cv=5).

5. Entraîner RandomForestClassifier(n_estimators=100, random_state=42). Comparer son exactitude à celle de l'arbre unique(Bonus)